l'école d'ingénierie informatique

COMPTE RENDU DE PROJET

DÉVELOPPEMENT WEB

Hôtel Neptune

Application web de gestion hôtelière

Implémentation en PHP, SQL, HTML & CSS

Wierez Mylo

B1 – 2025-26

Rendu le : 05/02/2026

Sommaire

Présentation 3
Thème 3
Objectifs 3
Analyse du sujet 3
Cahier des charges 3
Architecture de l’application 4
Méthodologie 4
Approche 4
Outils et ressources 4
Organisation du travail 5
Réalisation 5
Modélisation des données 5
Architecture du code 6
Explication des contrôleurs 6
Explication des vues 8
Analyse détaillée du code 9
Sécurité et gestion de session 10
Synthèse 11
Bilan général 11
Difficultés rencontrées 11
Limites identifiées 12
Améliorations possibles 12
Conclusion 12

Présentation

Thème

Ce projet consiste en la conception et le développement d'une application web complète pour l'Hôtel Neptune, réalisée dans le cadre du cours de développement web. Il s'inscrit dans une mise en situation professionnelle où nous incarnons un prestataire en développement web chargé de moderniser la gestion d'un hôtel souhaitant digitaliser ses réservations et sa gestion interne.

Les notions fondamentales mises en œuvre sont les suivantes :

- Architecture légère en PHP (séparation contrôleurs / vues)
- Manipulation de bases de données relationnelles via PDO (PHP Data Objects)
- Gestion des sessions PHP pour l'authentification des utilisateurs et des administrateurs
- Construction de requêtes SQL préparées pour prévenir les injections
- Conception de l'interface utilisateur en HTML5 et CSS3
- Traitement des formulaires et validation des données côté serveur

Objectifs

L'objectif fonctionnel est de livrer une application web opérationnelle permettant aux clients de l'hôtel de consulter les chambres disponibles, de filtrer selon leurs critères et d'effectuer des réservations en ligne, tandis que les administrateurs disposent d'un espace sécurisé pour gérer les chambres, les réservations et les profils clients.

L'objectif pédagogique est de maîtriser la construction d'une application web complète de bout en bout : de la modélisation de la base de données à l'interface utilisateur, en passant par la logique métier, la sécurité des accès et la gestion des erreurs.

Analyse du sujet

Cahier des charges

Le sujet nous demande de développer une application web en PHP, SQL, HTML et CSS pour l'Hôtel Neptune. L'application est divisée en deux parties distinctes :

La partie publique, accessible à tous, comprend :

- Une page d'accueil présentant l'hôtel
- La liste des chambres disponibles avec leurs caractéristiques
- La possibilité de filtrer les chambres selon le nombre de personnes et le budget
- La possibilité de réserver une chambre pour des dates données
- Des pages de connexion et d'inscription pour les clients

La partie privée (espace administrateur), accessible uniquement aux administrateurs authentifiés, comprend :

- La gestion des chambres (ajout, suppression)
- La gestion des réservations (validation et facturation)
- La gestion des utilisateurs et clients

Architecture de l'application

L'application adopte un modèle proche du patron MVC (Modèle-Vue-Contrôleur), même si elle ne l'implémente pas formellement. Les responsabilités sont clairement séparées en deux dossiers principaux :

- controller/ : contient la logique métier, les requêtes SQL et la gestion des sessions
- vue/ : contient les fichiers HTML/PHP d'affichage

Cette séparation facilite la maintenance et l'évolution du projet.

Méthodologie

Approche

Le développement a suivi une démarche itérative et progressive :

- Analyse du cahier des charges et modélisation de la base de données (tables user, admin, chambre, reservation).
- Mise en place des pages de base (accueil, liste des chambres) et de la structure de navigation.
- Implémentation de l'authentification utilisateur et des guards de session.
- Développement des fonctionnalités de réservation avec vérification de disponibilité.
- Construction de l'espace administrateur et des fonctions de gestion.
- Intégration du CSS et des scripts JavaScript, tests de bout en bout.

Outils et ressources

- Langages : PHP 8, SQL (MySQL), HTML5, CSS3, JavaScript
- Base de données : MySQL avec connexion via PDO
- Environnement : VS Code, PHPStorm, WAMP / XAMPP
- Documentation : php.net, developer.mozilla.org (MDN)
- Gestion de version : Git

Organisation du travail

Le projet a été structuré autour de deux dossiers principaux : hotel/ pour la version PHP opérationnelle et html/ pour les maquettes et prototypes HTML statiques. Cette approche a permis de valider l'interface avant d'implémenter la logique serveur.

Réalisation

Modélisation des données

1. La table user

Stocke les informations des clients de l'hôtel. Elle contient les champs suivants :

Champ	Type	Description
id_user	INT (PK)	Identifiant unique de l'utilisateur
login	VARCHAR	Nom d'utilisateur unique
email	VARCHAR	Adresse e-mail
password	VARCHAR	Mot de passe
num_tel	VARCHAR	Numéro de téléphone
adresse	VARCHAR	Adresse postale

2. La table chambre

Représente les chambres disponibles dans l'hôtel :

Champ	Type	Description
id_chambre	INT (PK)	Numéro unique de la chambre
taille	INT	Capacité (nombre de personnes)
nb_lit	INT	Nombre de lits
prix	FLOAT	Prix par nuit en euros
dispo	INT	Disponibilité de la chambre (0/1)

3. La table reservation

Enregistre les réservations effectuées par les clients :

Champ	Type	Description
id_user	INT (FK)	Référence au client
id_chambre	INT (FK)	Référence à la chambre
date_debut	INT	Date d'arrivée (timestamp Unix)
date_fin	INT	Date de départ (timestamp Unix)
nb_personne	INT	Nombre de personnes

4. La table admin

Stocke les comptes administrateurs, avec un matricule supplémentaire servant de deuxième facteur d'identification :

Champ	Type	Description
id_admin	INT (PK)	Identifiant unique de l'admin
matricule	VARCHAR	Code matricule (second facteur d'auth.)

Champ	Type	Description
login	VARCHAR	Identifiant de connexion
email	VARCHAR	Adresse e-mail
password	VARCHAR	Mot de passe

Architecture du code

Le projet est organisé selon la structure suivante :

Fichier / Dossier	Rôle
index.php	Point d'entrée de l'application, inclut la vue d'accueil
controller/	Contient toute la logique métier et les accès à la base de données
vue/	Contient toutes les pages HTML/PHP d'affichage (partie client)
vue/admin/	Vues réservées à l'espace administrateur
vue/ressources/	Images (logo), feuille de style CSS et scripts JavaScript

Explication des contrôleurs

Le dossier controller/ regroupe dix fichiers PHP, chacun responsable d'une opération métier spécifique :

Contrôleur	Rôle
login.php	Authentifie un client via login + mot de passe, démarre la session et redirige vers les chambres.
logout.php	Détruit la session PHP et redirige vers la page d'accueil.
signin.php	Crée un nouveau compte client après vérification de l'unicité du login.
filtre.php	Stocke les critères de filtrage (dates, convives, budget) en session et redirige vers la liste des chambres.
gestion_resa.php	Vérifie la disponibilité d'une chambre et crée la réservation en base de données.
guard.php	Protège les pages nécessitant une connexion client ; redirige vers l'accueil si non connecté.
login_admin.php	Authentifie un administrateur via login + mot de passe + matricule.
guard_admin.php	Protège les pages d'administration ; redirige si non connecté en tant qu'admin.
ajouter_chambre.php	Insère une nouvelle chambre en base de données après validation des données POST.
supprimer_chambre.php	Supprime une chambre de la base de données (avec gestion des erreurs de contrainte FK).
filtre_admin.php	Filtre l'affichage des chambres dans l'espace admin par numéro de chambre.

1. Contrôleur login.php

Ce contrôleur suit le patron suivant : récupération des données POST, connexion à la base via getDb(), préparation et exécution d'une requête SELECT avec paramètres liés, puis gestion du résultat (démarrage de session ou redirection).

2. Contrôleur gestion_resa.php

C'est le cœur de la logique de réservation. Il reçoit l'identifiant de la chambre via GET et les dates via POST. La fonction estOccupee() effectue une requête SQL couvrant tous les cas de chevauchement de créneaux :

```
(:date_debut >= date_debut AND :date_debut < date_fin) OR (:date_fin >
date_debut AND :date_fin <= date_fin) OR (:date_debut <= date_debut
AND :date_fin >= date_fin)
```

Si aucun chevauchement n'est détecté et que les dates sont valides, la fonction creationResa() insère la réservation en base.

Explication des vues

Le dossier vue/ contient sept fichiers, couvrant l'ensemble des pages client et admin :

Vue	Rôle
accueil.php	Page d'accueil publique de l'hôtel.
chambres.php	Liste les chambres disponibles. Intègre la logique de filtrage via les variables de session et affiche une modale de filtrage.
page_chambre.php	Page de détail d'une chambre avec formulaire de réservation (dates, nombre de personnes).
login.php	Formulaire de connexion client (login + mot de passe).
signin.php	Formulaire d'inscription client (login, email, mot de passe, téléphone, adresse).
admin/accueil_admin.php	Page de connexion administrateur (login + mot de passe + matricule).
admin/gestion_chambres.php	Interface d'administration des chambres : liste, ajout et suppression.
admin/ gestion_reservations.php	Interface de consultation et gestion des réservations.
admin/client.php	Consultation des profils clients par l'administrateur.

Analyse détaillée du code

1. La fonction getDb()

Chaque contrôleur définit une fonction getDb() qui retourne une instance PDO. Ce choix, bien que répétitif, garantit l'indépendance de chaque fichier. PDO est préféré à mysqli car il supporte plusieurs systèmes de bases de données et impose l'usage de requêtes préparées :

```
function getDb(): PDO {      try {          $db = new
PDO('mysql:host=localhost;dbname=hotel;charset=utf8', 'root', 'password');
return $db;      } catch (Exception $e) {          die('Erreur de connexion :
' . $e->getMessage());      } }
```

2. Requêtes préparées et protection contre les injections SQL

Toutes les interactions avec la base de données utilisent des requêtes préparées via $pdo->prepare() et $stmt->execute(). Les paramètres utilisateur ne sont jamais concaténés directement dans les chaînes SQL, ce qui protège l'application contre les injections SQL :

```
$sql = 'SELECT * FROM user WHERE login = :login AND password = :password';
$stmt = $pdo->prepare($sql); $stmt->execute(['login' => $login, 'password'
=> $password]);
```

3. Gestion du filtrage des chambres

Le filtre est géré via les variables de session. Lorsque l'utilisateur soumet le formulaire, filtre.php stocke les critères en $_SESSION et redirige vers chambres.php. Cette vue lit alors l'état du filtre et appelle soit rechercherChambres() soit getAllChambres() :

```
if (isset($_SESSION['filter']) && $_SESSION['filter']) {      $chambres =
rechercherChambres($_SESSION['convives'], $_SESSION['max_price']); } else {
$chambres = getAllChambres(); }
```

Sécurité et gestion de session

1. Le système de guards

Deux fichiers de garde assurent la protection des ressources sensibles :

- guard.php : vérifie que $_SESSION["is_logged"] est à true avant d'afficher toute page nécessitant une connexion client. En cas d'absence, l'utilisateur est redirigé vers l'accueil.
- guard_admin.php : même logique pour l'espace administrateur, vérifie $_SESSION["is_logged_admin"].

Ces fichiers sont inclus en tête de chaque contrôleur et vue protégée via require, garantissant qu'aucun accès direct n'est possible sans authentification préalable.

2. Double authentification administrateur

La connexion administrateur exige trois éléments : un login, un mot de passe et un matricule. La requête SQL vérifie simultanément ces trois champs, ajoutant une couche de sécurité supplémentaire par rapport à la connexion client classique. Un intrus connaissant le login et le mot de passe ne pourrait pas accéder à l'espace admin sans le matricule.

3. Gestion des erreurs de contrainte

Lors de la suppression d'une chambre possédant des réservations, la base de données renvoie une erreur de contrainte de clé étrangère (code SQL 23000). Le contrôleur supprimer_chambre.php capture cette exception PDO et affiche un message d'erreur explicite au lieu de laisser l'application planter.

Synthèse

Bilan général

Le projet remplit son objectif principal : livrer une application web fonctionnelle pour l'Hôtel Neptune, couvrant l'ensemble du cahier des charges. Les fonctionnalités de consultation des chambres, de filtrage, de réservation et d'administration sont opérationnelles.

Les concepts clés du développement web vus en cours sont mis en pratique de façon concrète : connexion base de données via PDO, requêtes préparées, gestion de sessions PHP, séparation des responsabilités entre contrôleurs et vues, et protection des accès par guards. La sécurité a été une priorité à chaque étape : aucune donnée utilisateur n'est injectée directement dans les requêtes SQL.

Difficultés rencontrées

- La gestion correcte des chevauchements de dates de réservation, qui nécessite de couvrir trois cas distincts dans la requête SQL.
- La synchronisation de l'état du filtre entre les requêtes, notamment la réinitialisation propre de la session lors du retrait du filtre.
- La gestion des cas limites lors de la suppression de chambres possédant des réservations actives.
- La prévention des accès non autorisés aux pages d'administration via les guards, notamment la séquence correcte de session_start() et des includes.

Limites identifiées

- Les mots de passe sont stockés en clair dans la base de données. Une application en production devrait utiliser password_hash() et password_verify().
- La fonction getDb() est dupliquée dans chaque contrôleur. Une classe de singleton ou un fichier de configuration partagé éliminerait cette redondance.
- Certaines fonctions de contrôleur utilisent encore var_dump() pour les messages d'erreur, ce qui n'est pas adapté à un contexte de production.
- Aucune validation côté serveur n'est effectuée sur le format des dates ou la valeur numérique du nombre de personnes.

Améliorations possibles

- Hashage des mots de passe avec password_hash() / password_verify() pour une sécurité conforme aux standards.
- Centralisation de la connexion base de données dans un fichier db.php inclus une seule fois.
- Remplacement des var_dump() par un système de gestion des erreurs et des messages flash via la session.
- Ajout d'une validation des formulaires côté serveur (format des dates, plages numériques, longueur des champs).
- Mise en place d'une gestion des rôles plus fine pour l'administration (niveaux de droits).
- Intégration d'un système de facturation générant des PDF de confirmation de réservation.
- Ajout d'un calendrier interactif pour la sélection des dates de séjour.

Conclusion

Ce projet constitue une application complète et concrète des concepts fondamentaux du développement web abordés en cours. Il démontre qu'il est possible de construire une application web fonctionnelle, sécurisée et structurée en utilisant des outils simples et académiques : PHP, SQL, HTML et CSS.

L'Hôtel Neptune dispose désormais d'une solution digitale permettant à ses clients de consulter et réserver des chambres en ligne, et à son équipe administrative de gérer l'ensemble des opérations depuis un espace sécurisé. La séparation entre contrôleurs et vues offre une base de code maintenable et évolutive.

La méthode adoptée — analyse du cahier des charges, modélisation des données, implémentation incrémentale par fonctionnalité, puis tests — s'est révélée efficace et reproductible. Les limites identifiées, notamment la gestion des mots de passe et la redondance du code de connexion, constituent des pistes d'amélioration claires pour une version de production.